

# CS & IT ENGINEERING

## Compiler Design



Intermediate code and optimization

Lecture No. - 01



By- Venkat sir





# Recap of Previous Lecture



Topic

Synthesized attribute

Topic

Inherited attribute

Topic

S-attributed Definition

Topic

L-attributed definition

Translation Schemes



# Topics to be Covered



Topic

Translation scheme

Topic

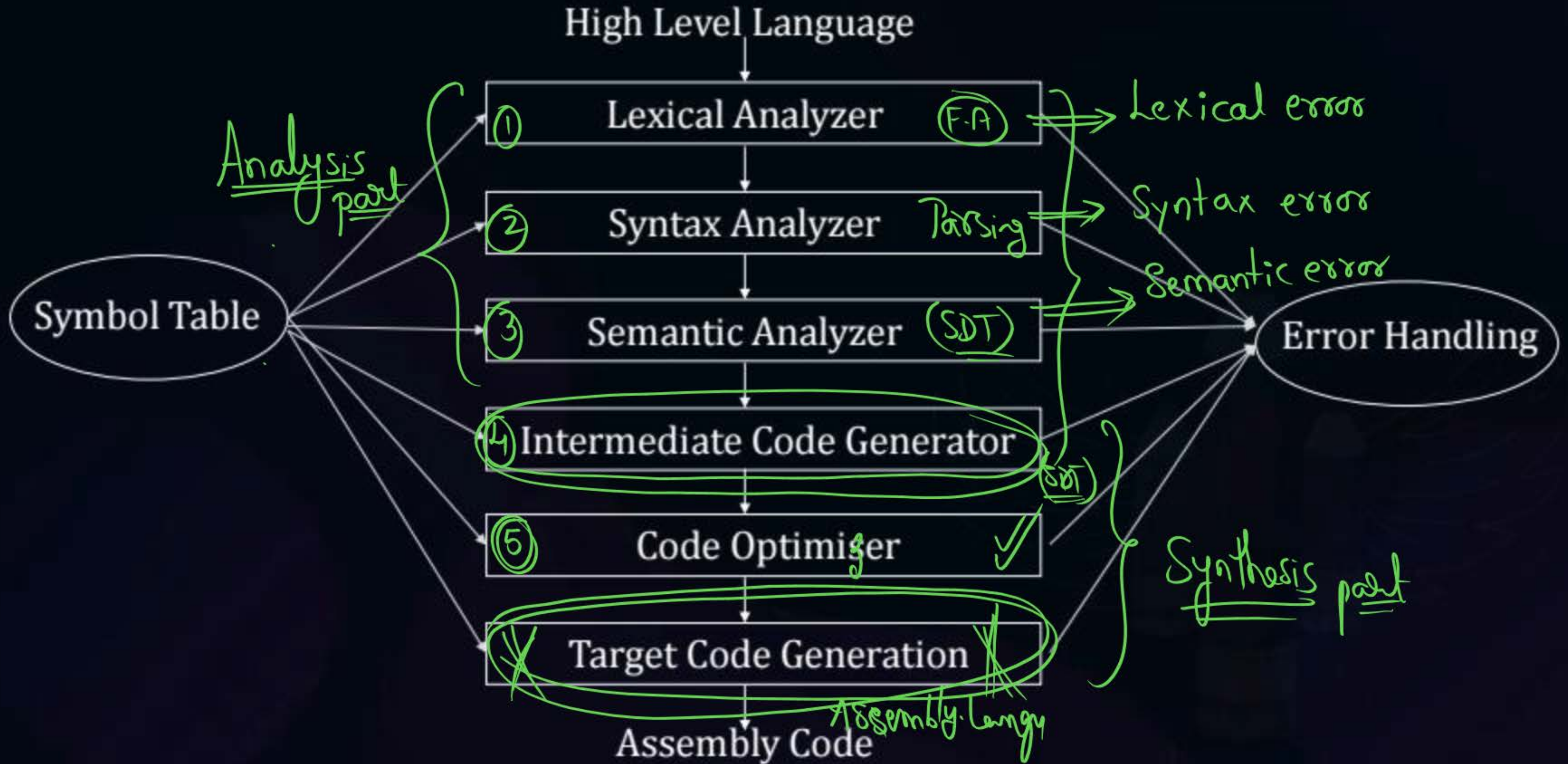
Intermediate Code generation





# Topic : Introduction of Compiler Design

int a, b, c, d;  
a = b + c;







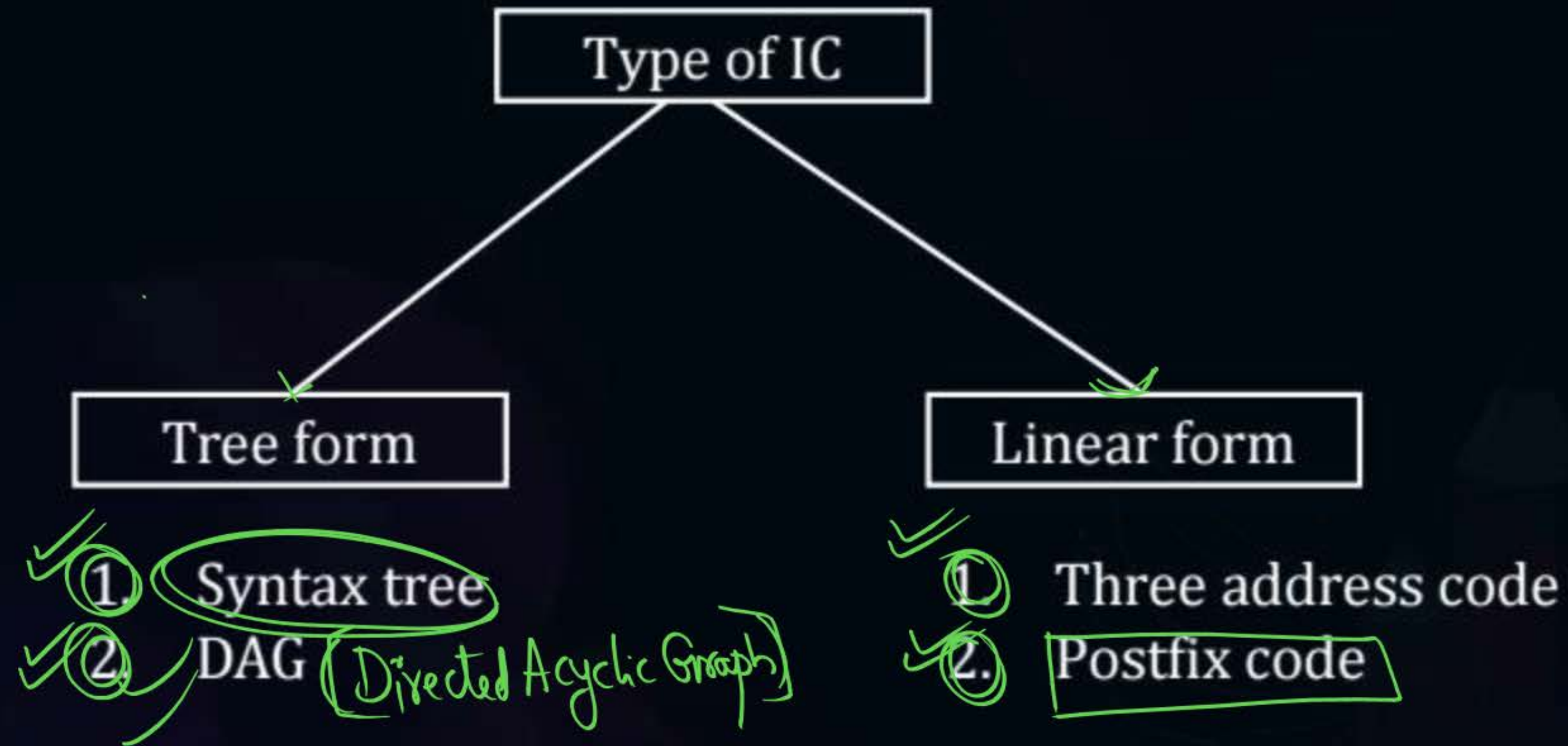
## Topic : Intermediate Code Generation

### Advantage:

- ✓ 1. Easy to generate from source code
- ✓ 2. Easy to convert to target code
- ✓ 3. Optimization is easy
- ✓ 4. Retargeting is possible (if is generating)



## Topic : Intermediate Code Generation







## Topic : Intermediate Code Generation

- ① Expression statement
- ② Flow control statement {for, while, if else}
- ③ Declaration statement int a, b, c;
- ④ Input and output statement printf, scanf
- ⑤ Procedure/functions



## Topic : Syntax tree



1. ✓ Optimized form of parse tree is syntax tree
2. ✓ Syntax tree is constructed for expression
3. ✓ Syntax represents order of evaluation of expression
4. ✓ In syntax tree for an expression all operand present at leaf position and all operator present at root.

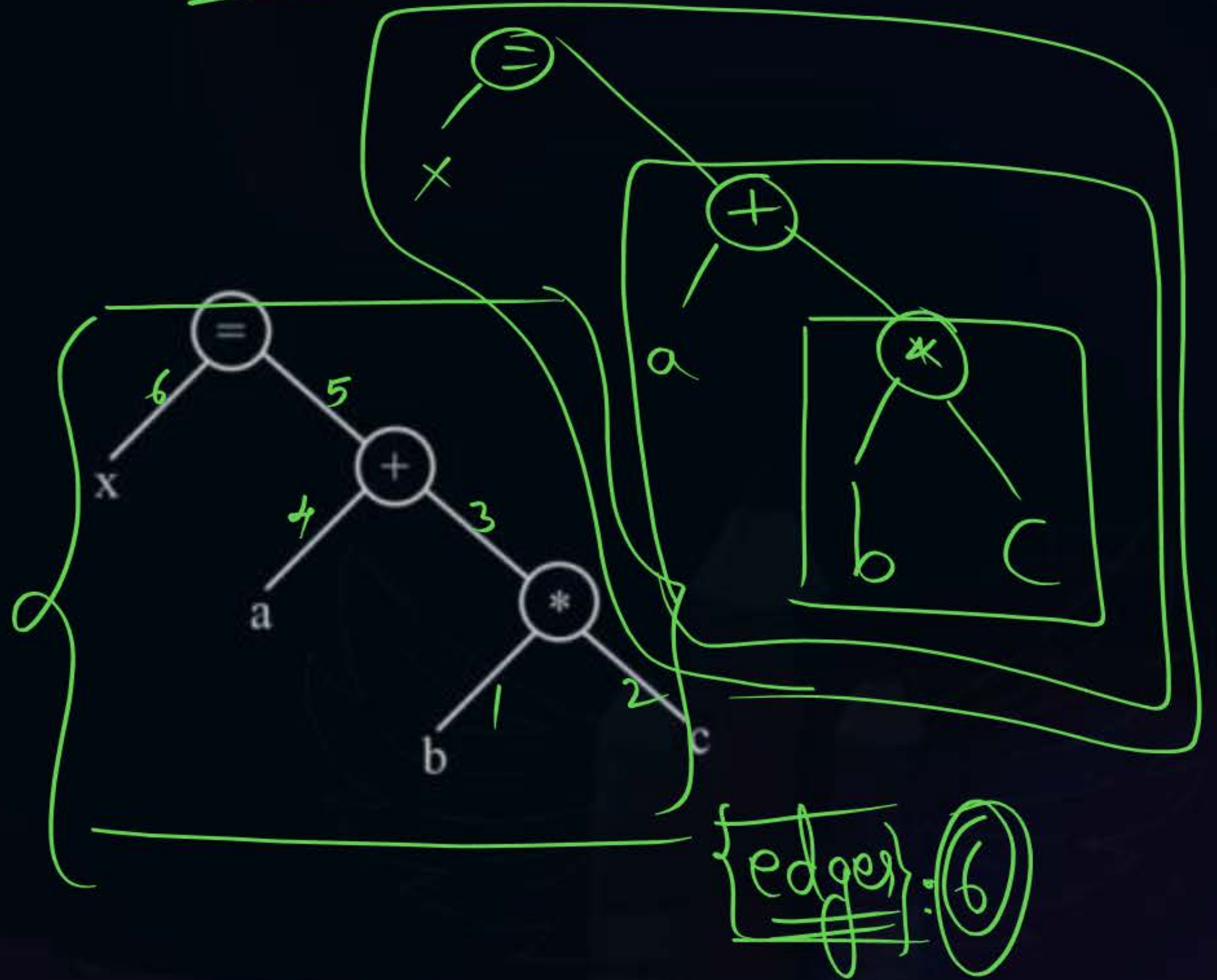
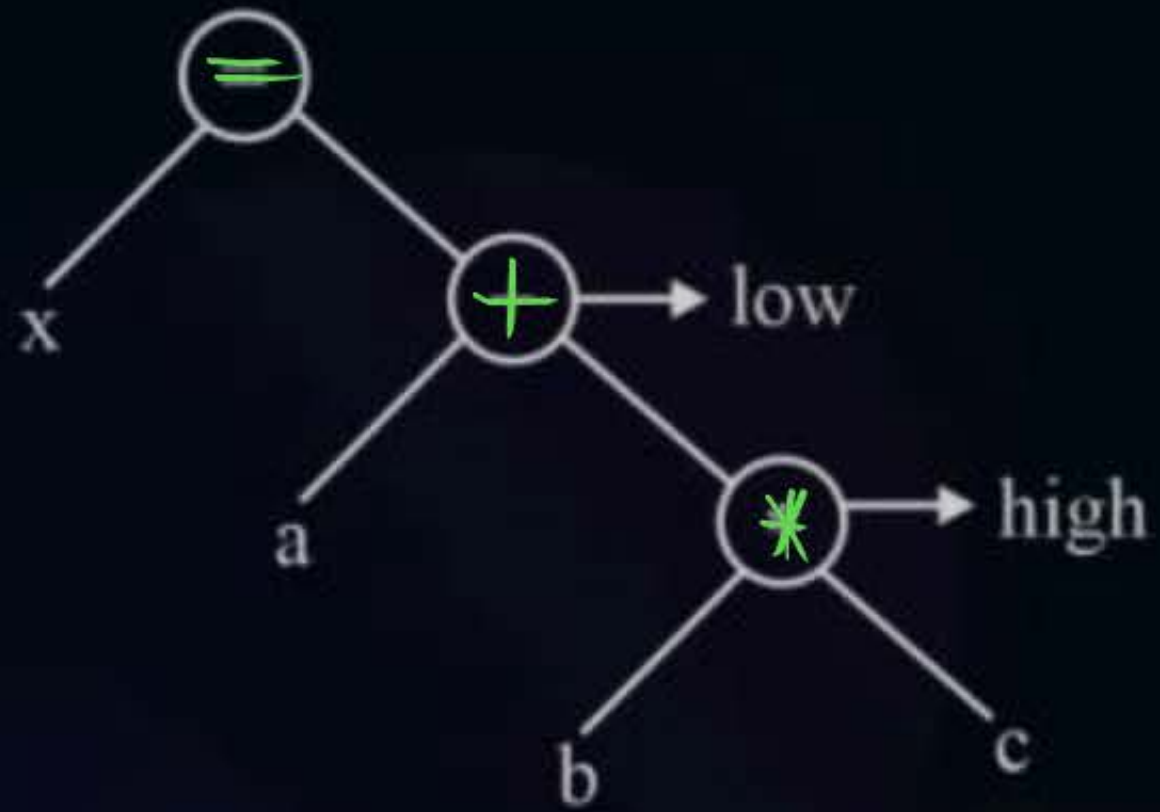




# Topic : Syntax tree

Syntax tree

Syntax tree :  $x = a + b * c$

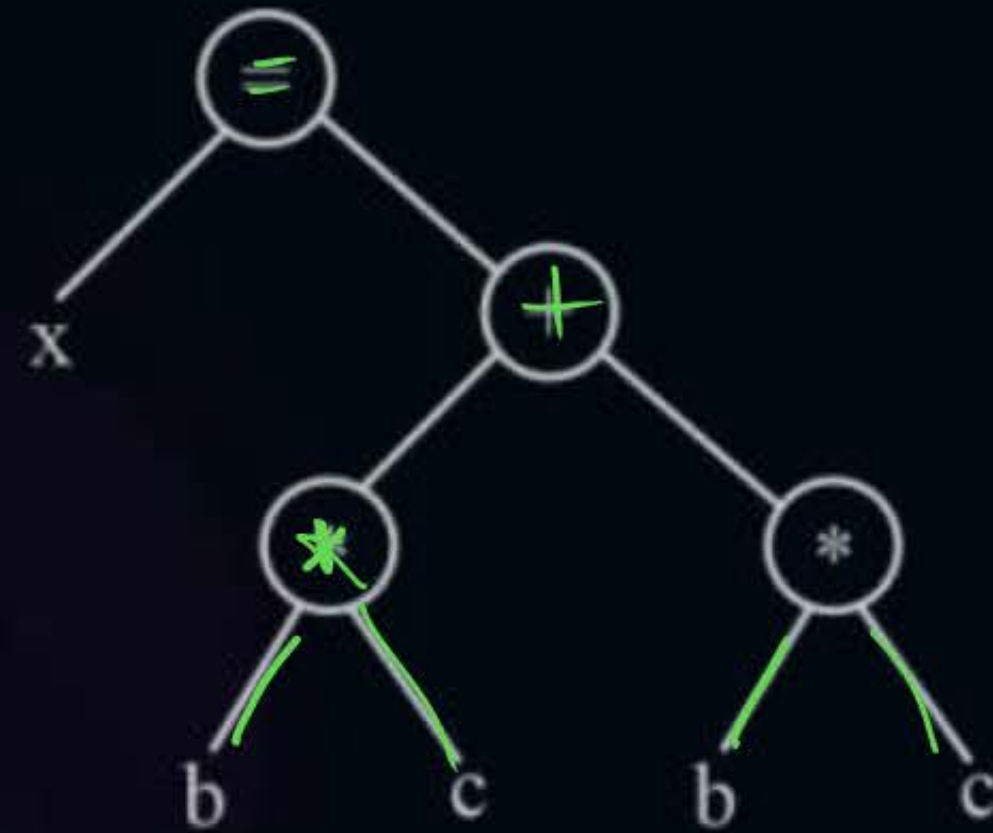




## Topic : Syntax tree

1.

$x = b * c + b * c$



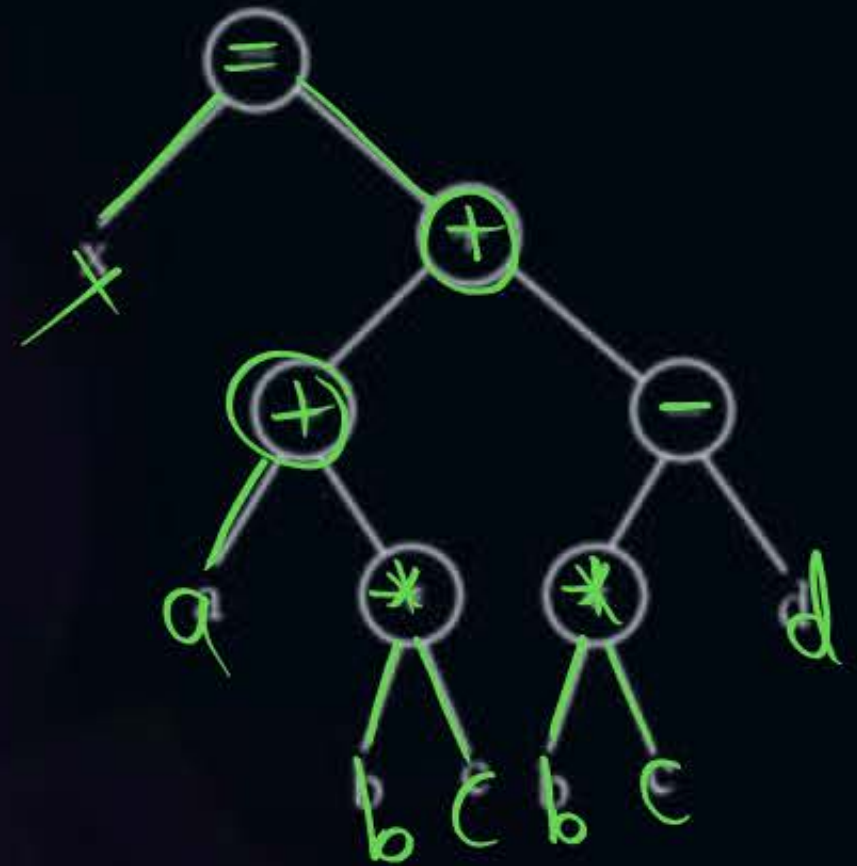
Common sub expressions  
exist in syntax tree





## Topic : Syntax tree

2.  $x = (a + b * c) + (b * c - d)$  } Number of edges in syntax tree?

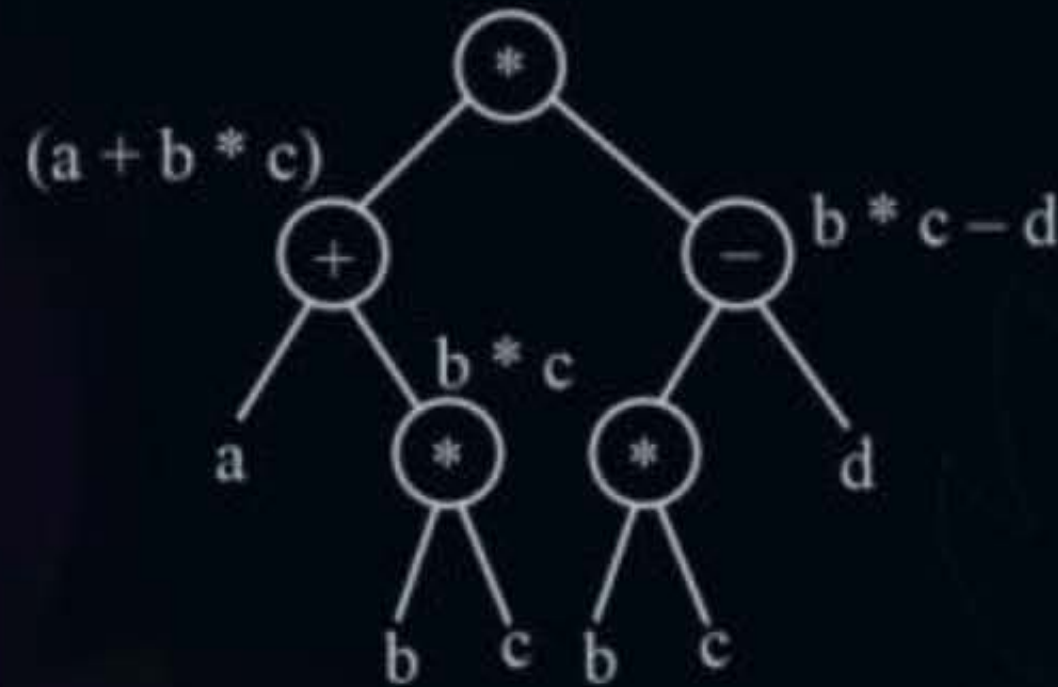




## Topic : Syntax tree

3. What is the expression for the given syntax tree

$(a + b * c) * (b * c - d)$







## Topic : Syntax tree

4. Consider the following operators

$\{\uparrow \rightarrow \downarrow\}$

$\uparrow \rightarrow$  right allocative

$\downarrow \rightarrow$  left associative

Construct syntax tree for

$7 \downarrow 3 \uparrow 4 \uparrow 3 \downarrow 2$

Construct syntax tree for

$7 \downarrow 3 \uparrow 4 \uparrow 3 \downarrow 2$





## Topic : Intermediate Code Generation

### **Directed Acyclic Graph (DAG)**

It is a syntax tree in which common sub expression are removed.

#### **Advantage:**

Common sub expression elimination optimization is performed using DAG.

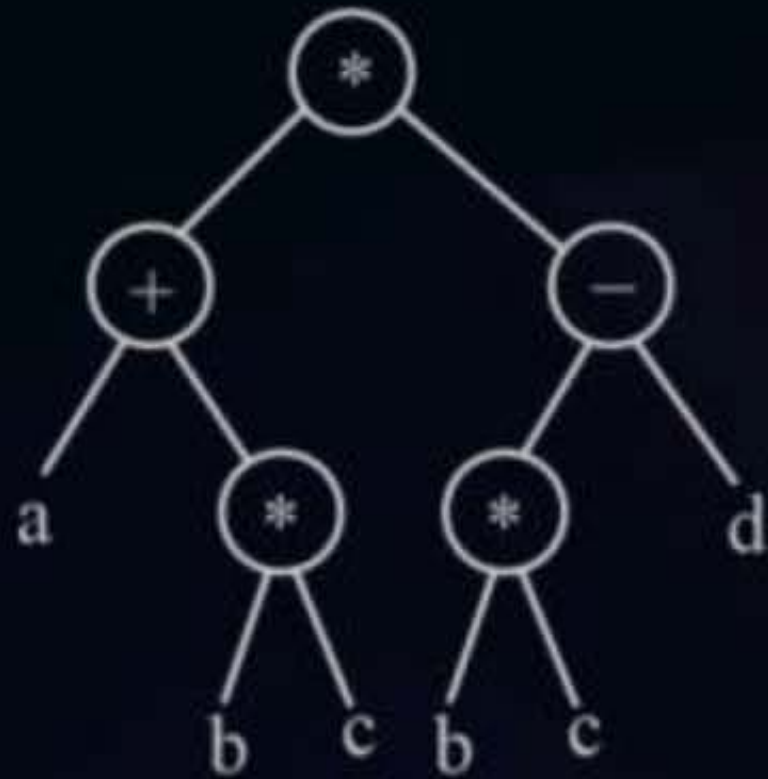




# Topic : Intermediate Code Generation

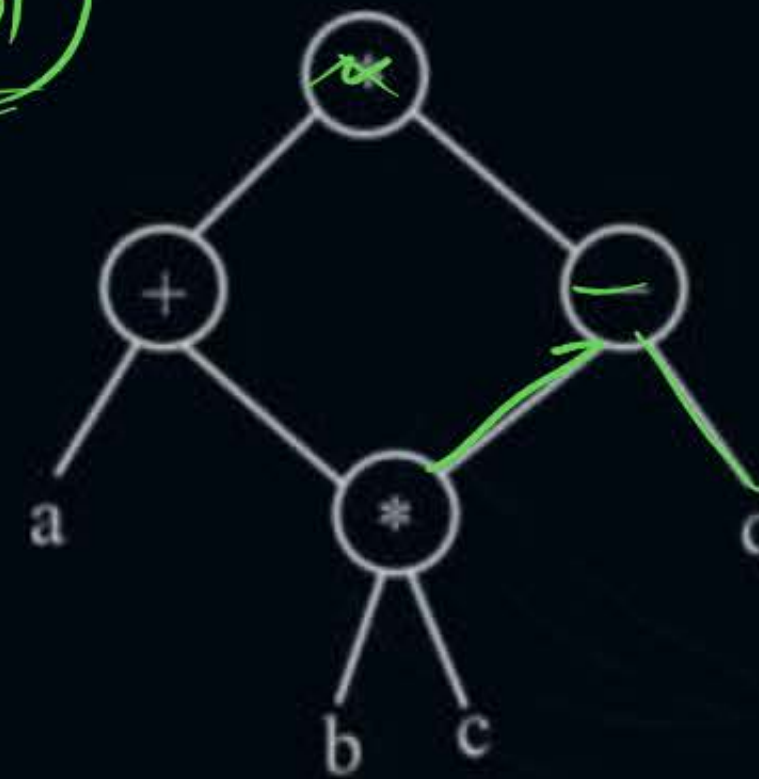
## Syntax tree

$(a + b * c) * (b * c - d)$



## Directed Acyclic Graph (DAG)

(DAG)



edges?

Q) How many edges present in DAG for the following expression



$$(a+b) * (a+b) * (a+b)$$

DAG



edges = ?

6 edges



(Q) How many edges present in DAG

$a + a + a + a + a$

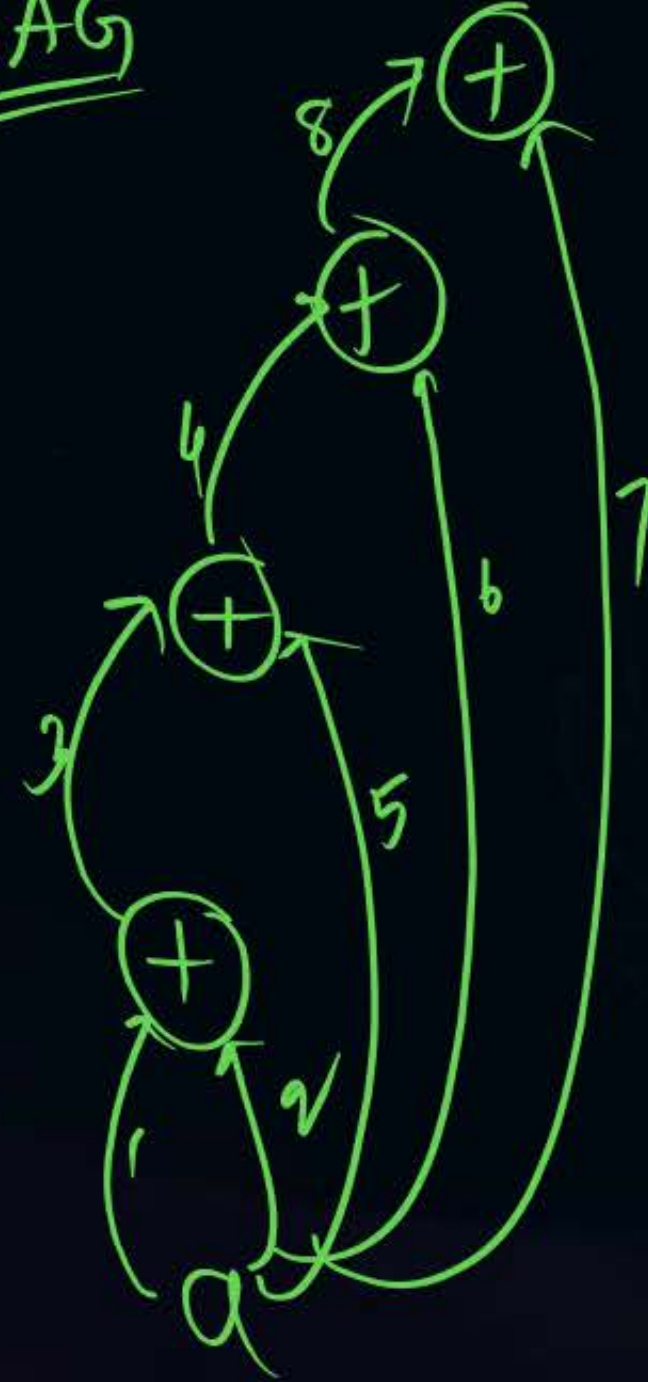
$a + a$

$a + a$

DAG



DAG

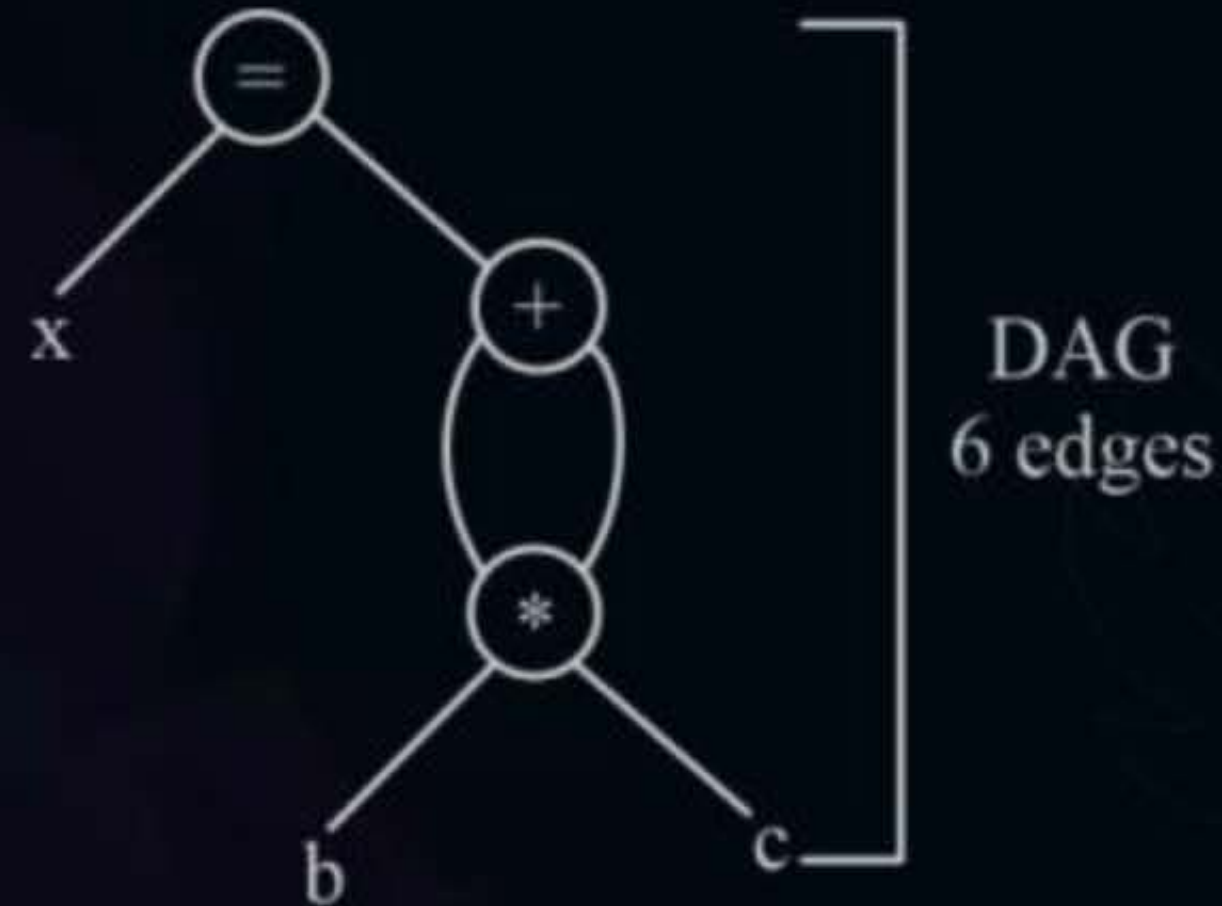




## Topic : Intermediate Code Generation

1. Construct DAG for  $x = b * c + b * c$

how many edges present in the DAG







## Topic : Intermediate Code Generation

2. DAG for  $x = a + a + a + a + a$

Final total number of nodes and edges present in the DAG?

DAG



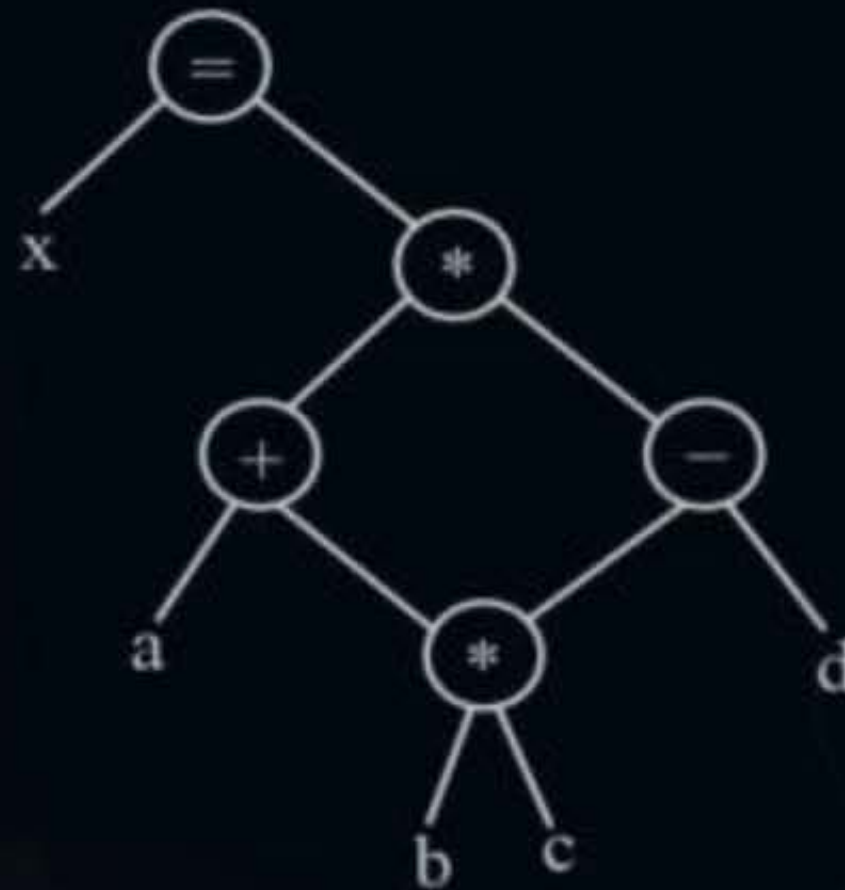
Edges : 10 edges

Nodes : 7 nodes



## Topic : Intermediate Code Generation

3.  $x = (a + b * c) * (b * c - d)$  total number of nodes and edges.



Edges : 10

Nodes : 10

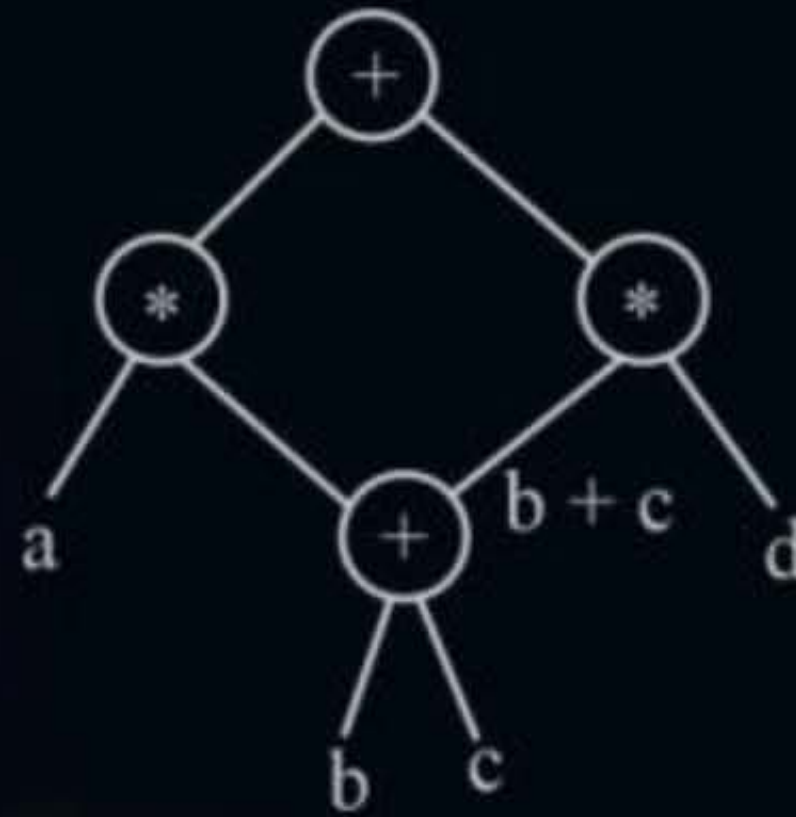




## Topic : Intermediate Code Generation

4. Construct expression for given DAG

DAG



**Expression**

$[a * (b + c) + [(b + c) * d]]$



## Topic : Intermediate Code Generation

ADD R<sub>0</sub>, R<sub>1</sub>

### Three Address Code:

TAC is an intermediate code in which high level language statement can be represented by using atmost 3 address

↓

ADD

↓

R<sub>01</sub>

↓

R<sub>1</sub>







## Topic : Intermediate Code Generation

1.  $x = yopz$
2.  $X = opY$
3.  $x = y$
- } Expression
- 
4.  $x[i] = y$
- $x = y[i]$
- } arrays
- 
5. Goto L } unconditional jump
6. 

If a > b goto  $L_1$

goto  $L_2$

 } Conditional jump
- } Flow control
-



## Topic : Intermediate Code Generation

### 7. Procedure/function

|   |                                |   |
|---|--------------------------------|---|
| [ | Param $P_1$                    | ] |
|   | Param $P_2$                    |   |
|   | .                              |   |
|   | .                              |   |
|   | .                              |   |
|   | Param $P_n$                    |   |
| [ | call name, number of arguments | ] |





## Topic : Intermediate Code Generation

1.  $x = a + b * c$

TAC

|                 |
|-----------------|
| $t_1 = b * c$   |
| $t_2 = a + t_1$ |
| $x = t_2$       |

$$\begin{aligned}t_1 &= b * c \\t_2 &= a + t_1 \\x &= t_2\end{aligned}$$



## Topic : Intermediate Code Generation

2.  $x = (a * b) + (c * d) / (a * b)$

How many temporary variable generated

$$t_1 = a * b$$

$$t_2 = c * d$$

$$t_3 = a * b$$

$$t_4 = t_2 / t_3$$

$$t_5 = t_1 + t_4$$

$$x = t_5$$

Common Sub expressions exist in TAC





## Topic : Intermediate Code Generation

3.  $x = a * b + c * d - e * f$

How many temporary variable generated to construct TAC.

$$t_1 = a * b$$

$$t_2 = c * d$$

$$t_3 = e * f$$

$$t_4 = t_2 + t_3$$

$$t_5 = t_4 - t_3$$

$$x = t_5$$

} Temp

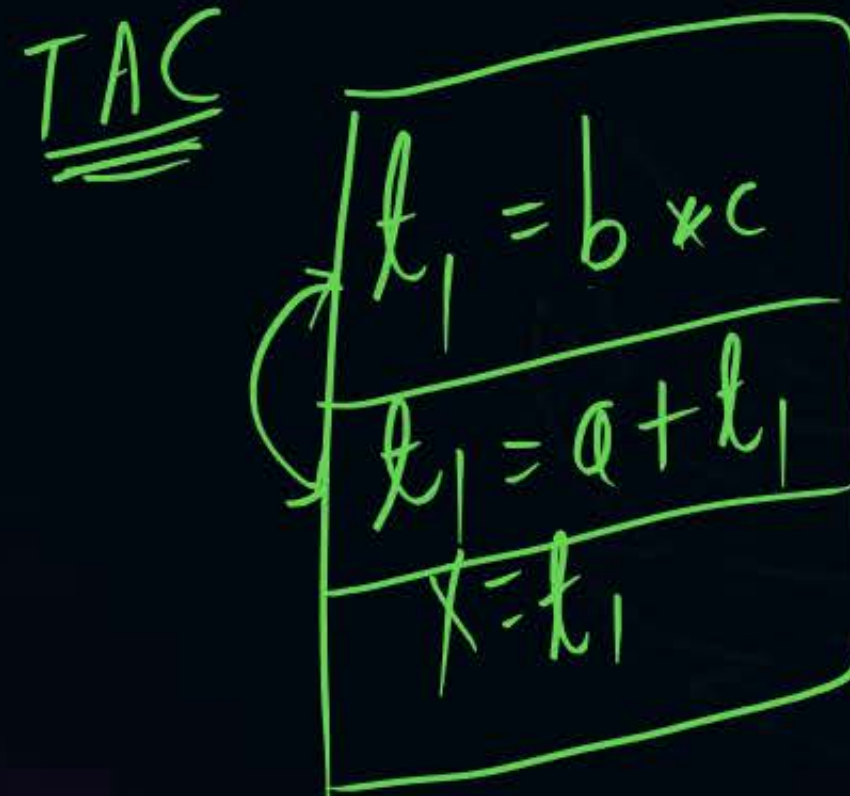
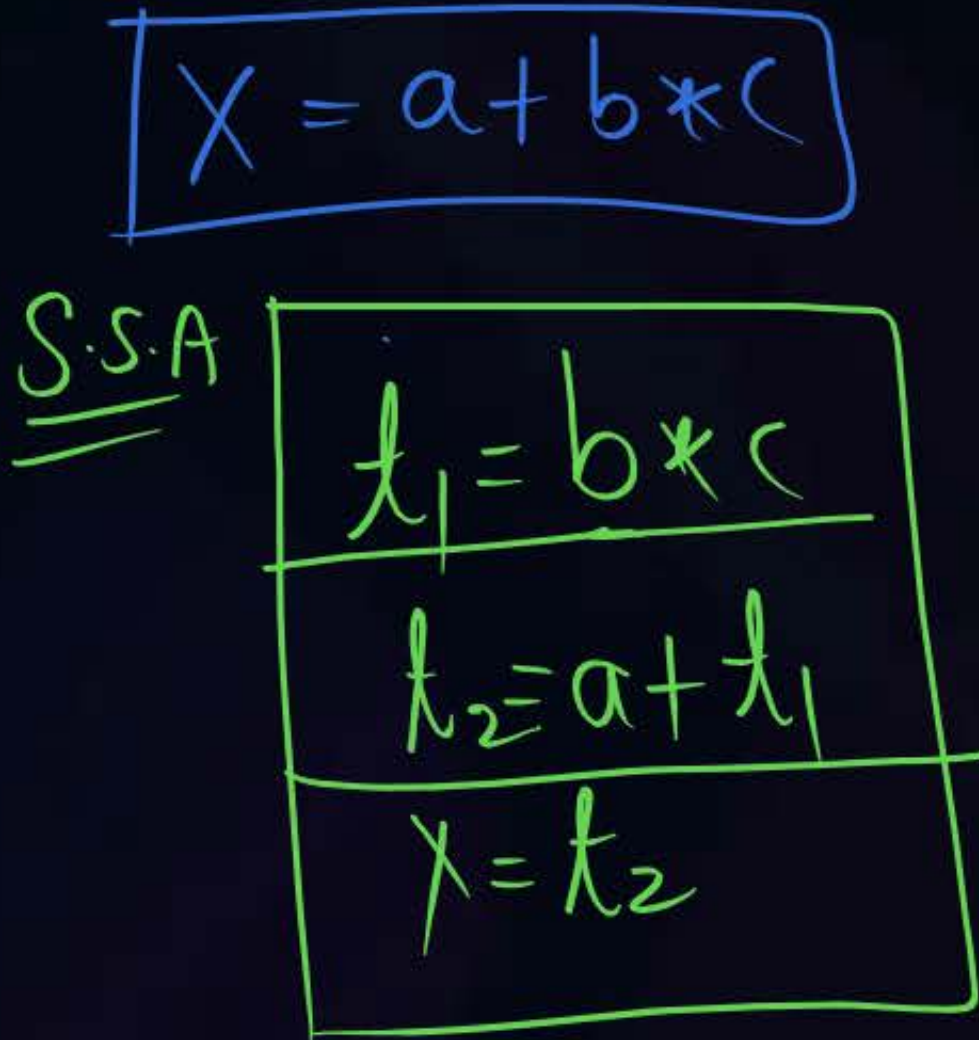


## Topic : Intermediate Code Generation

### Static Single Assignment: (TAC)

In static single assignment every expression is assigned to new variable.

Reusing of variable <sup>value</sup> is not allowed in static single assignment.







## Topic : Intermediate Code Generation

```
if (a > b)
{
    x = y + z;
}
else
{
    A = B + C;
}
```

LAST

{ If  $a > b$  goto  $L_1$  }  
goto  $L_2$

$L_1$ :  $t_1 = y + z$   
 $x = t_1$   
goto LAST

$L_2$ :  $t_1 = B + C$   
 $A = t_1$

LAST:



## Topic : Intermediate Code Generation

### Flow Control

if  $a > b$  goto  $L_1$

goto  $L_2$

$L_1$ :  $t_1 = y + z$

$x = t_1$

$L_2$ :  $t_1 = B + C$

$A = t_1$

goto LAST





## Topic : Intermediate Code Generation

#Q. Construct TAC for following for Loop.

```
for (i = 1; i < 100; i + + )
```

```
{
```

```
    x = y + z * a
```

```
}
```



## Topic : Intermediate Code Generation

TAC

$i = 1$  ✓

$L_0$ : if ( $i < 100$ ) goto  $(L_1)$

goto Last

$(L_1)$ :  $t_1 = \underline{A} * A$

$t_2 = y + t_1$

$x = t_2$

$t_3 = i + 1$

$i = t_3$

goto  $L_0$

LAST: }

for ( $i=1$ ;  $i < 100$ ;  $i++$ )

{  
     $x = y + z * A$   
}





## 2 mins Summary



Topic

One

Lexical Analysis

Topic

Two

Parsing

Topic

Three

SDT

Topic

Four

I.C.G.

Topic

Five



211212112

ACBCBCBC

**THANK - YOU**